Week 10

C237:
Software Application Development

LESSON 20:
Deployment

Software Development Life Cycle (SDLC)

What is SDLC?

- **Software Development Life Cycle (SDLC)** is a structured process that guides the planning, development, testing, deployment, and maintenance of software applications.
- **Why** use SDLC:
  - Improves software quality
  - Helps teams work in an organized manner
  - Reduces errors and rework
  - Makes projects easier to manage

Key Phases of SDLC

- **Planning:**
  - Decide the purpose, target users and features of the application.
- **Design:**
  - Plan the pages, user interface, database and application flow.
- **Development**
  - Build the application by writing and integrating code.
- **Testing:**
  - Check that the application works correctly and fix any issues.
- **Deployment:**
  - Publish the application so that users can access it online.
- **Maintenance:**
  - Improve the application by fixing bugs and adding enhancements.

Key Phases of SDLC

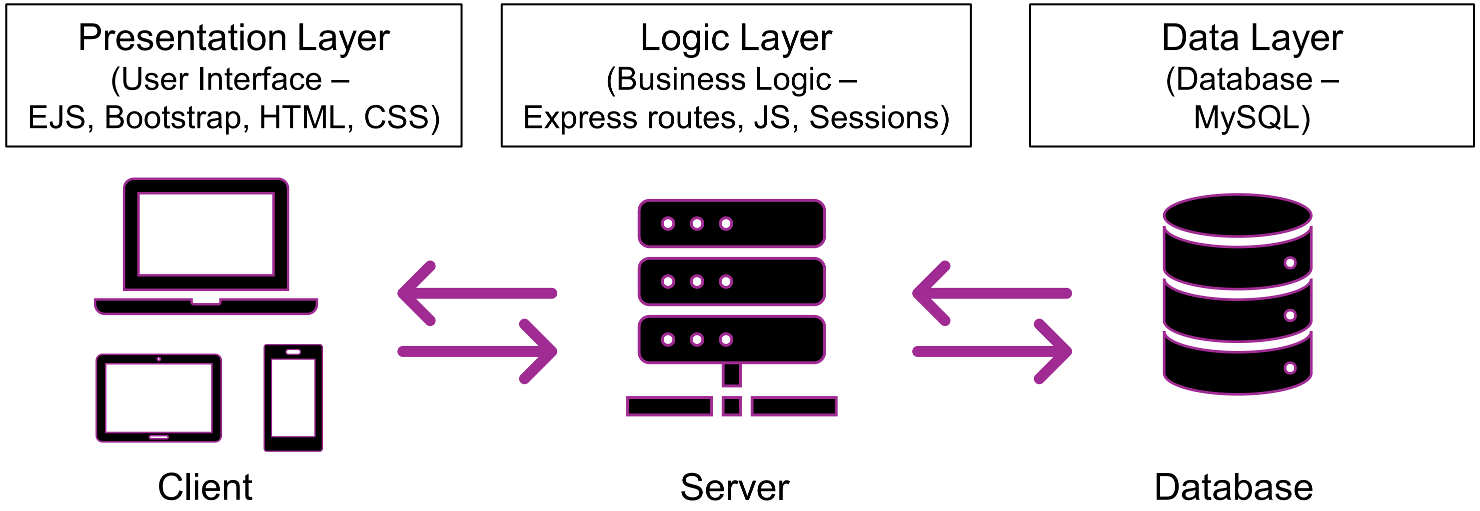
SDLC Phase	In C237
Planning	Choose application idea
Design	Plan pages, routes, database
Development	Build Express & MySQL app
Testing	Test every feature
Deployment	Deploy to Render + Azure
Maintenance	Fix bugs and add enhancements

Why Deployment Matters?

- Makes your application **accessible online** so others can use it.
- **Allows users to access your application from anywhere**, not just your own computer.
- **Makes it easier to test and demonstrate** your application
- **Prepares your application for real-world use** and future enhancements.

3-Tier Architecture

- Separates a web application into **three layers**, each with a different responsibility.



- Each layer has its **own responsibilities**
- **Benefits:**
  - Keeps the user interface, application logic and database separate.
  - Makes the application easier to develop and debug.
  - Allows each layer to be modified independently.

Localhost vs Remote host

Localhost	Remote Host
Runs on your own computer	Runs on another computer or server
Only you can access it	Can be accessed by other users over the Internet
Used during development	Used after deployment
Example: `http://localhost:3000`	Example: Render, Azure

- During development, you use localhost.
- After deployment, users access your application through a remote host.
- **In C237**
  - **Render** hosts your **web application**
  - **Azure** MySQL hosts your **database**.

Committing a project To GitHub

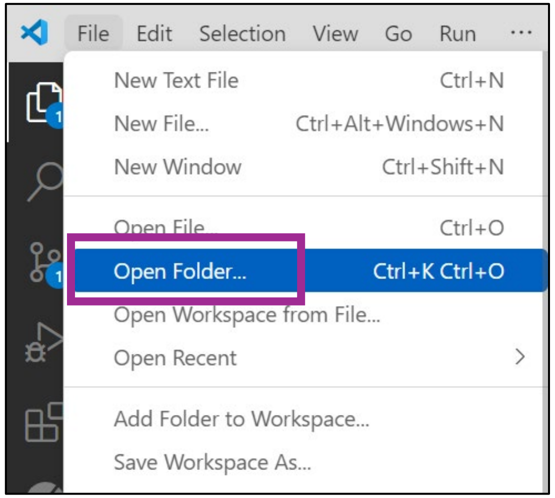
GitHub

- **GitHub** is a web-based platform for hosting and collaborating on code using **version control** (Git).

- **Why** Github?
  - **Version control**
    - helps teams work together, review updates, and maintain a clean code history.
  - **Commits** record changes, allowing developers to:
    - Track progress
    - Revert to earlier versions
    - Understand how the project evolves

Publish RegistrationApp to GitHub

- Create a folder **L20** in your **C237 folder**.
- Copy the **registrationApp** folder from your “L20 Activity Files” and paste it into the **L20 folder** you created earlier.(Alternatively, if your **registrationApp from L19** is working correctly, you may use that instead.)
- Open **registrationApp** in VS Code.



- You have initialized and commit a Node.js application project folder on GitHub before. If you need reference, you may refer to **L12 - GitHub_VSCode_Guide.pdf** in L12.

Database Hosting

What is Database Hosting?

- Database hosting is a service that provides a **server or cloud platform** for storing and managing databases online.
- **Microsoft Azure MySQL** is the recommended database hosting platform for this module. It allows your application to connect to an online MySQL database from anywhere.

- **Alternative Hosting Options**
  - If you prefer, you may also use other MySQL hosting providers such as **filess.io** or similar free hosting services. However, please note that free services may have limitations such as reduced reliability, storage limits, or service availability.

Let’s try - Database Hosting

- To host your database on Azure, follow the guide provided in your **L20 Activity Folder**.
  - Refer to **Student Guide to connect to Azure MySQL using MySQL Workbench.pdf** found in your L20 Activity Folder.
- Connection Details

Field	What to enter
Connection Name	C237 Database
Connection Method	Standard (TCP/IP)
Hostname	(Provided by your lecturer)
Port	3306
Username	(Provided by your lecturer)
Password	Click **Store in Vault**... and enter the password provided by your lecturer

Database Hosting

- Now that you are done with hosting your database online, let's update your **app.js** database connection credentials.
- Open your **registrationApp** folder in VS Code and update your app.js connection credentials:

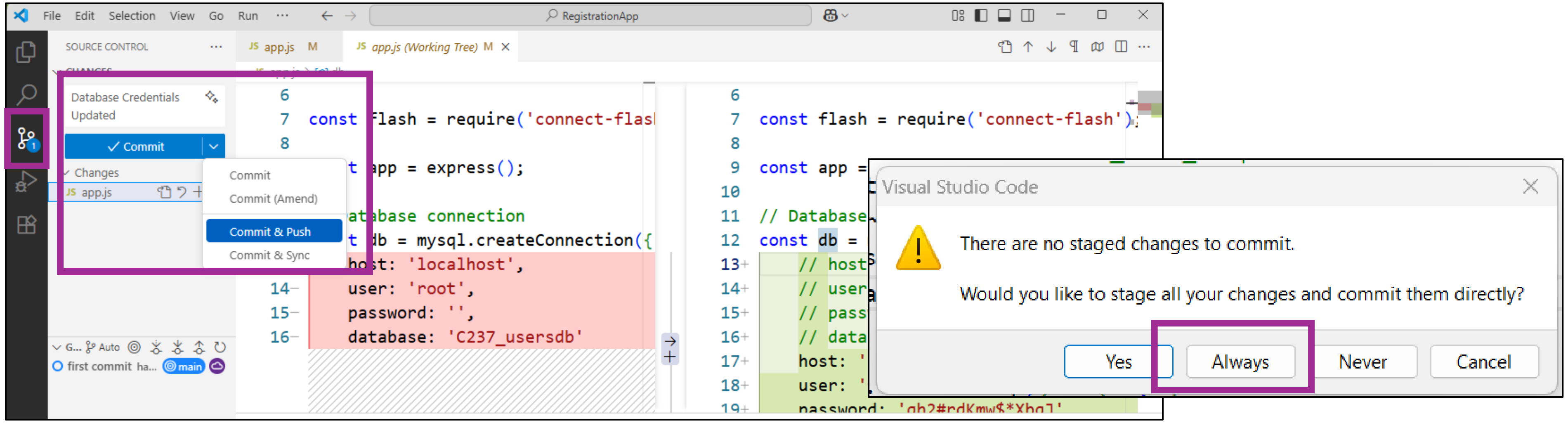
```
// Database connection
const db = mysql.createConnection({
    host: '<provided by lecturer>.mysql.database.azure.com',
    user: '<provided by lecturer>',
    password: '<provided by lecturer>',
    database: '<team database name>'
});

// const db = mysql.createConnection({
//     host: 'localhost',
//     user: 'root',
//     password: '',
//     database: 'C237_usersdb'
// });
```

Test your connection

- Save all your files.
- Open your terminal and enter “**npx nodemon app.js**” (if your server is not already running.)
- Open your browser and enter the url http://localhost:3000 and test database connection by:
  - Registering **2 new users** (**1 admin**, **1 user**)
  - **Login to both users** and make sure they are directed to their respective pages
    - **Admin** should be directed to **admin dashboard**
    - **User** should be directed to **user dashboard**.

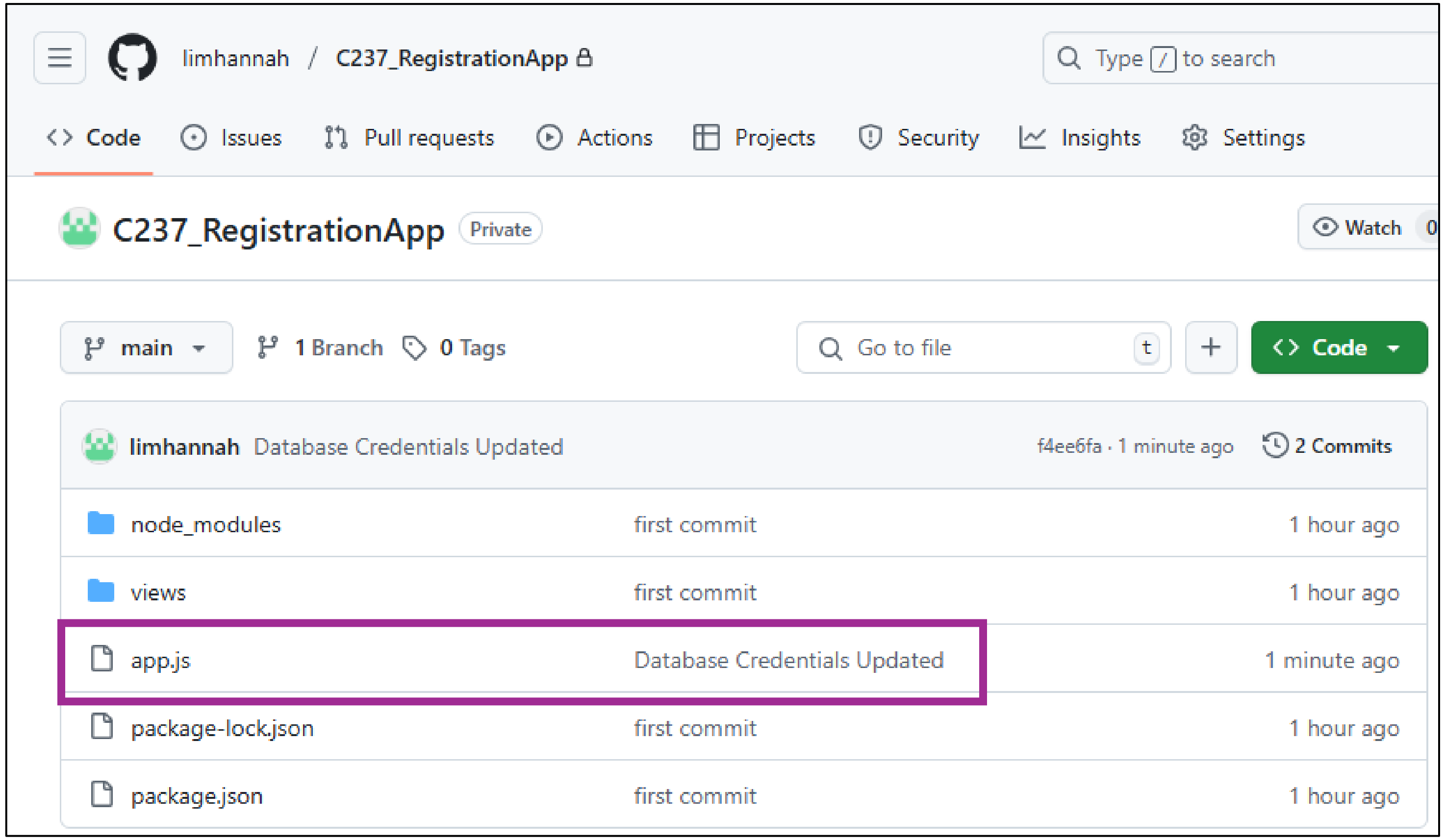
Database Hosting

- After updating the database credentials, a notification icon with the number 1 should appear in your source control tab.
- Click on the Source Control tab, then click on the filename ‘**app.js**’. You should see the updates you made.
- In the **message box** above the commit button, enter "**Database Credentials Updated**".
- Click on the “Commit & Push” Button.



Database Hosting

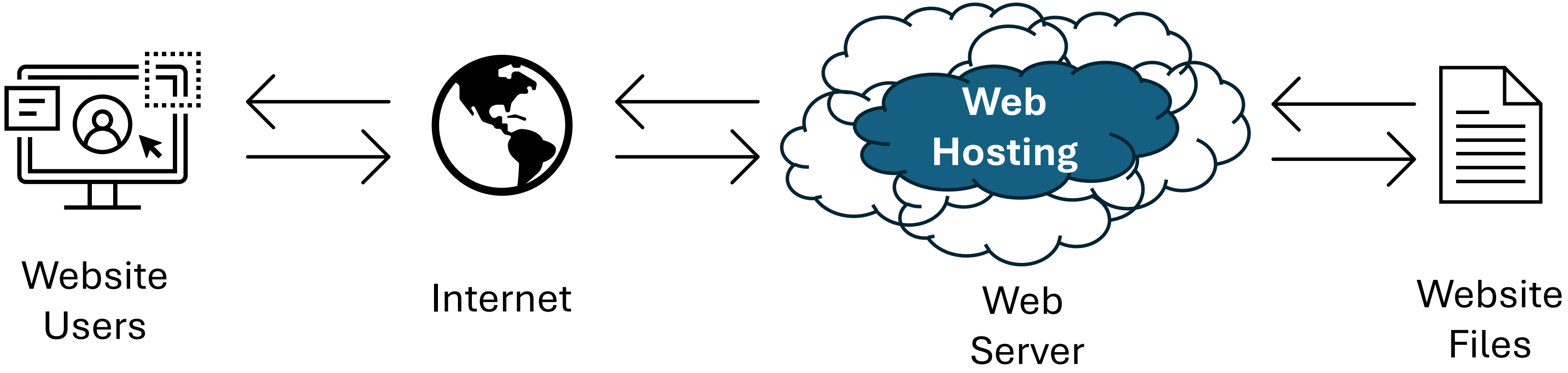
- You can verify that your updates have been successfully committed by checking on GitHub.



Web Hosting

What is Web Hosting?

- Web hosting allows your **web application to be accessible online**
- It stores your application on a **server** so users can access it through a web browser
- In this module, **Render** is the recommended platform for hosting your Node.js application.
- After deployment, anyone with the URL can access your application (subject to your application's permissions)



Web Hosting Companies

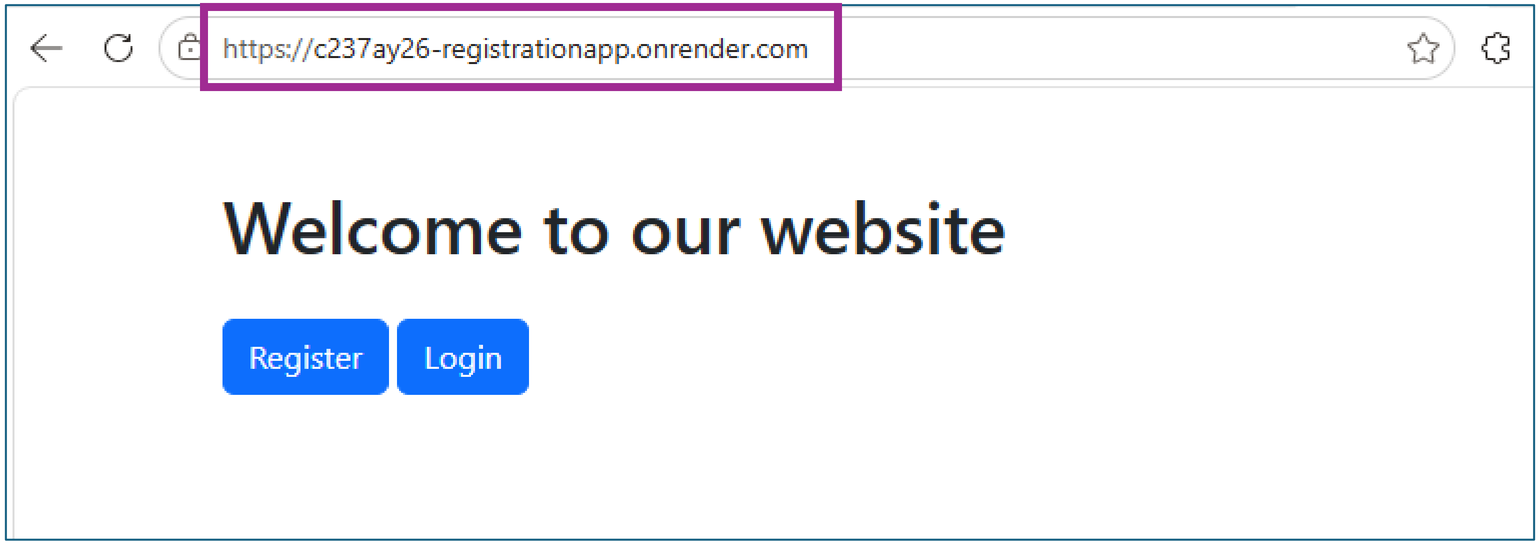
- Some **examples** of **hosting companies** using cloud infrastructure:
  - Amazon Web Services (AWS)
  - Google Cloud Platform (GCP)
  - Heroku
  - Microsoft Azure
  - Render
- Most providers offer multiple **pricing tiers**
- Ranges from:
  - Basic plans (for personal projects & small websites)
  - To advanced plans (for high-traffic or enterprise-level apps)

Web Hosting on Render

- Easy-to-use platform for **hosting web apps**, including **Node.js**
- **Simplifies deployment** – no need to manage servers manually
- Supports **multiple languages** and frameworks (not just Node.js)
- Offers a **free tier**:
  - Great for **hobby projects** and **student apps**
  - Sufficient for testing and basic development

Hosting a NodeJS App on Render

- **Deploying to Render**
  - Open **“Guide to hosting a NodeJS app on Render.pdf”** (in your L20 Activity Files folder)
  - After deployment, click the **URL on the top left** of your Render dashboard
  - **Congrats!** Your registration app project is now live



Activity

L20 Activity – Supermarket App Setup

- Copy the **SupermarketApp folder** from your **L20 Activity Files**
  → Paste it into your own **L20 folder**
- Import database
  → Use **supermarketdb.sql** in localhost MySql Workbench
- Open your **SupermarketApp in VS Code.**
- Run Supermarket App
  → enter "**npx nodemon app.js**" (if your server is not already running.)
- Launch App
  → http://localhost:3000

L20 Activity – Test the App

- Register **2 new users**
  - **1 admin**,
  - **1 user**
- **Login** and check. Make sure they are directed to their respective pages
  - **Admin** should be directed to **Inventory Page**
  - **User** should be directed to **Shopping Page**
- The SupermarketApp is currently running on localhost with local database.

L20 Activity - Deployment

1. **Commit and Push to GitHub**
   - Use VS Code to commit and push your app to GitHub
2. **Deploy MySQL to Azure**
   - Open **MySQL Workbench** and connect to Azure
   - Create a new MySQL database on **Azure – e.g** c237_0XX_team1supermarketdb
   - **Import** the **supermarketdb.sql** file into your own team's database (remember to remove the Create database statements)
3. **Deploy to Render**
   - Go to render.com
   - Create a new Web Service
   - Connect to your GitHub repo
4. **Update DB Connection in app.js**
5. **Commit and Push updated code to GitHub**

CA2 Reminder

- Full Assignment Specification was uploaded to PoliteMall in **Week 9**.
- Please refer to the full **CA2 specification PDF** for complete instructions and requirements
- You should start working on your CA2 with your teammates now.
  - Decide on your application idea
  - Start planning and developing your application
  - Be prepared to show your work to your lecturer in **L21**

- **Due:** By **24 Jul 2026, 2359hrs**
- **Submit to: SA3.0**

Deliverables

1. **Supermarket app Render**
2. **CA2 Interim Submission Check**
   - Add your lecturer as one of your github collaborator: <email given by lecturer>
   - **Team Development Journal**
     - **Section A – Team Information and**
     - **Section C - Team Contributon Record**

Upload all deliverables to POLITEMall **before Saturday, 2359hrs**

What have you learnt?

- Software Development Life Cycle (SDLC)
  - What is SDLC?
  - Key Phases of SDLC
- Deployment
  - Why Deployment?
  - Deploying a Project
- GitHub
  - Repositories and Version Control
  - Commits and Change Tracking
- Database Hosting
- Web Hosting

End of Lesson 20